OAVS TGT COMPUTER TEACHER EXAM

MCQ Practice Book

Comments in C

Complete Question Bank with Explanations & Answer Key

110 Multiple Choice Questions

Single-line Comments · Multi-line Comments · Compiler Behaviour · Syntax & Errors

Prepared: May 2026

TABLE OF CONTENTS

1.	About This Book	3
2.	Quick Reference: Comments in C	3
3.	Section A: Single-line Comments (Q1–Q12)	4
4.	Section B: Multi-line Comments (Q13–Q24)	5
5.	Section C: Uses of Comments (Q25–Q34)	6
6.	Section D: Compiler Behaviour (Q35–Q44)	7
7.	Section E: Syntax & Tricky Questions (Q45–Q66)	8
8.	Section F: Advanced & Mixed (Q67–Q110)	10
9.	Practice Test 1 (Q1–Q25) — 30 minutes	13
10.	Practice Test 2 (Q26–Q50) — 30 minutes	14
11.	Practice Test 3 (Q51–Q75) — 30 minutes	15
12.	Practice Test 4 (Q76–Q110) — 35 minutes	16
13.	Answer Key — All Sections	17
14.	Answer Key — Practice Tests	18
15.	Explanations Appendix	18

ABOUT THIS BOOK

This MCQ practice book is the **12th instalment** of the OAVS Computer Teacher (TGT) Exam Preparation Series. It covers the topic **Comments in C** comprehensively, with **110 Multiple Choice Questions** organised into thematic sections, four timed practice tests, complete answer keys, and detailed explanations.

Exam	OAVS Computer Teacher (TGT)
Topic	Comments in C (Prompt 12)
Total MCQs	110
Sections	6 thematic + 4 practice tests
Difficulty	Easy → Moderate → Advanced
Standards Covered	C89, C99, C11 comments

QUICK REFERENCE: COMMENTS IN C

Feature	Single-line (//)	Multi-line (/* */)
Standard	C99 onwards	C89 onwards
Syntax	// text to end of line	/* text spanning multiple lines */
Nesting	Not applicable	NOT supported
Best Use	Short inline notes	Function headers, block disable
Compiler Action	Stripped — replaced by space	Stripped — replaced by space
Line Continuation	Yes (with backslash \)	Not needed — spans naturally
MISRA-C Preferred	No	Yes

Key Rule: Comments are replaced by a single space by the C preprocessor and generate NO machine code. Nested /* */ is NOT allowed in standard C.

SINGLE-LINE COMMENTS

Q1. Which symbol is used to begin a single-line comment in C?

A) #
B) //
C) /*
D) --

Answer: (B)

Explanation: In C, '//' starts a single-line comment; everything after it on that line is ignored by the compiler.

Q2. What is the scope of a single-line comment in C?

A) Until the next semicolon
B) Until the closing */,
C) Until the end of the current line
D) Until the end of the function

Answer: (C)

Explanation: A '//' comment extends from '//' to the end of that line only.

Q3. Which of the following is a valid single-line comment in C?

A) /* This is a comment */
B) // This is a comment
C) # This is a comment
D) -- This is a comment

Answer: (B)

Explanation: // is the correct syntax for single-line comments in C.

Q4. What will be the output of the following code? #include int main(){ int x=5; //x=10; printf("%d",x); return 0; }

A) 10
B) 5
C) Compilation error
D) Undefined behaviour

Answer: (B)

Explanation: The assignment x=10 is inside a comment, so x retains its value 5.

Q5. Single-line comments in C were introduced in which standard?

A) C89/ANSI C
B) C99
C) C11
D) C17

Answer: (B)

Explanation: // single-line comments were introduced in the C99 standard (ISO/IEC 9899:1999).

Q6. Which of the following is NOT a valid way to comment out a single line in C?

A) // int x = 5;
B) /* int x = 5; */
C) # int x = 5;
D) Both B and C are valid

Answer: (C)

Explanation: # is a preprocessor directive marker, not a comment symbol in C.

Q7. In C, can a single-line comment appear after a statement on the same line?

A) No
B) Yes
C) Only in C++
D) Only in C11

Answer: (B)

Explanation: Single-line comments can appear anywhere on a line, including after a statement. e.g.: int x = 5; // initialise x

Q8. What happens if '//' appears inside a string literal?

A) It starts a comment
B) It is treated as part of the string
C) Compilation error
D) Undefined behaviour

Answer: (B)

Explanation: Inside double quotes, '//' has no special meaning; it is just two characters in the string.

Q9. How many single-line comments can appear in one C program?

A) Maximum 100
B) Maximum 255
C) Unlimited
D) Only 1 per function

Answer: (C)

Explanation: There is no limit on the number of comments in a C program.

Q10. What will the following print? printf("Hello // World");

A) Hello
B) Hello // World
C) Nothing after //
D) Compilation error

Answer: (B)

Explanation: '// World' is inside a string literal, so it is printed as-is.

Q11. Which of the following correctly places a single-line comment?

A) int a = 5 // value;
B) int a = //5; 5;
C) //int a = 5;
D) Both A and C

Answer: (D)

Explanation: Both 'int a = 5 // value;' and '//int a = 5;' are valid placements of '//' comments.

MULTI-LINE COMMENTS

Q12. Which symbols are used to start and end a multi-line comment in C?

A) // and //
B) /* and */
C) { and }
D)

Answer: (B)

Explanation: Multi-line comments in C start with / and end with */.*

Q13. A multi-line comment in C is also called a:

A) Shell comment
B) Block comment
C) Inline comment
D) Preprocessor comment

Answer: (B)

Explanation: / ... */ comments are commonly called block comments because they can span multiple lines.*

Q14. Can multi-line comments be nested in C?

A) Yes, unlimited nesting
B) Yes, up to 10 levels
C) No, they cannot be nested
D) Yes, but only in C11

Answer: (C)

Explanation: C does not support nested / */ comments. A */ inside a comment closes the first /*, causing unexpected behaviour.*

Q15. What is the output of the following code? int x = /* 5 */ 10; printf("%d", x);

A) 5
B) 10
C) 55
D) Compilation error

Answer: (B)

Explanation: The comment / 5 */ is removed by the preprocessor, so x = 10.*

Q16. Which of the following multi-line comment structures will cause an error in C?

A) /* Hello World */
B) /* Hello World */
C) /* Hello /* inner */ World */
D) /* */

Answer: (C)

Explanation: Nested / */ comments are not supported; /* Hello /* inner */ closes the comment before 'World */', causing a syntax error.*

Q17. What happens if a multi-line comment is not closed?

A) The rest of the file is treated as comment
B) Only the current function is affected
C) The compiler ignores the error
D) A warning is issued but code compiles

Answer: (A)

Explanation: An unclosed / will cause everything till the end of file (or a later */) to be treated as a comment, resulting in compilation errors.*

Q18. Which standard first included multi-line comments (/* */) in C?

A) C99
B) C11
C) C89 (ANSI C)
D) C17

Answer: (C)

Explanation: / */ block comments have been part of C since the original C89/ANSI C standard.*

Q19. A multi-line comment can span how many lines?

A) Up to 5 lines
B) Up to 100 lines
C) Any number of lines
D) Up to the file size limit

Answer: (C)

Explanation: A / ... */ comment can span as many lines as needed until */ is found.*

Q20. What will be the output? /* int x=5; int y=10; */ printf("Hello");

A) 5 10 Hello
B) Hello
C) Compilation error
D) Undefined

Answer: (B)

Explanation: The variable declarations are inside a block comment and are completely ignored.

Q21. Which of the following is a valid multi-line comment?

A) /* This is line1 This is line2 */
B) // This is line1 // This is line2
C) Both A and B
D) Neither A nor B

Answer: (C)

Explanation: Both are valid: (A) uses block comment syntax and (B) uses two consecutive single-line comments.

Q22. What is the minimum valid multi-line comment in C?

A) /
B) /**/
C) /* */
D) Both B and C

Answer: (D)

*Explanation: /**/ and /* */ are both valid (empty) multi-line comments.*

Q23. Inside a multi-line comment, what happens to the code written in it?

A) It is executed
B) It is ignored by the compiler
C) It causes warnings
D) It is stored in memory

Answer: (B)

Explanation: All content between / and */ is discarded by the compiler during the preprocessing phase.*

USES OF COMMENTS

Q24. Which of the following is the primary purpose of comments in C?

A) To increase execution speed
B) To document and explain code
C) To allocate memory
D) To define macros

Answer: (B)

Explanation: Comments are used to make code more readable and to explain the logic to other programmers.

Q25. Commenting out code during debugging is used to:

A) Permanently delete code
B) Temporarily disable code without deleting it
C) Speed up compilation
D) Allocate extra memory

Answer: (B)

Explanation: Commenting out code allows a programmer to disable portions temporarily for testing.

Q26. Which comment style is preferred for function documentation in C?

A) Single-line //
B) Block comment /* ... */
C) Both are equally preferred
D) Neither; documentation is separate

Answer: (B)

Explanation: Block comments are traditionally preferred for documenting functions, their parameters, and return values.

Q27. Comments in C are processed by:

A) The linker
B) The loader
C) The preprocessor/compiler (and stripped out)
D) The CPU

Answer: (C)

Explanation: Comments are removed during the preprocessing/lexical analysis phase and do not generate any machine code.

Q28. Which of the following is NOT a valid use of comments in C?

A) Describing the algorithm
B) Hiding sensitive data at runtime
C) Noting author and date
D) Explaining complex logic

Answer: (B)

Explanation: Comments are stripped by the compiler and never present at runtime, so they cannot hide runtime data.

Q29. Using comments to explain the purpose of a variable is called:

A) Debugging
B) Inline documentation
C) Preprocessing
D) Profiling

Answer: (B)

Explanation: Short comments next to variable declarations are a form of inline documentation.

Q30. Which tool commonly uses specially formatted comments to auto-generate documentation?

A) make
B) Doxygen
C) gdb
D) valgrind

Answer: (B)

Explanation: Doxygen reads specially formatted / ... */ comments to generate HTML/PDF documentation.*

Q31. What is a 'TODO comment'?

A) A comment marking code to delete
B) A comment noting a task still to be done
C) A compiler directive
D) A preprocessor command

Answer: (B)

Explanation: TODO comments (e.g., // TODO: fix this) are annotations reminding the programmer of pending work.

Q32. Comments are often used to preserve:

A) Old or alternate versions of code
B) Variable values at runtime
C) Stack frames
D) File headers only

Answer: (A)

Explanation: Programmers often comment out old code to keep it as a reference while testing new code.

Q33. Which of the following best describes self-documenting code?

A) Code with maximum comments
B) Code with clear variable/function names reducing the need for comments
C) Code without any comments
D) Code generated automatically

Answer: (B)

Explanation: Self-documenting code uses meaningful names so that comments are needed only for non-obvious logic.

COMPILER BEHAVIOUR WITH COMMENTS

Q34. At which stage does the C compiler remove comments?

A) Linking
B) Assembly
C) Preprocessing/Lexical analysis
D) Code optimisation

Answer: (C)

Explanation: Comments are stripped during the early preprocessing or lexical analysis stage, before parsing.

Q35. After removing a comment, the compiler replaces it with:

A) Nothing (empty string)
B) A single space character
C) A newline
D) A null byte

Answer: (B)

Explanation: According to the C standard, a comment is replaced by a single space during translation.

Q36. What does the compiler do with the text inside a comment?

A) Compiles it separately
B) Ignores it completely
C) Generates warnings about it
D) Stores it in the symbol table

Answer: (B)

Explanation: The compiler completely ignores any text inside comments; no code is generated from it.

Q37. Do comments affect the size of the compiled (binary) executable?

A) Yes, they increase it
B) Yes, they decrease it
C) No, they are stripped before compilation
D) Depends on compiler flags

Answer: (C)

Explanation: Comments are removed during preprocessing, so they have no effect on the binary size.

Q38. Which of the following will cause a compilation error related to comments?

A) // unclosed comment
B) /* unclosed comment
C) // nested // comment
D) /* empty comment */

Answer: (B)

Explanation: An unclosed / comment will cause the compiler to treat the rest of the file as a comment, typically resulting in errors.*

Q39. What is the output? int a = 10//* comment */2; printf("%d",a);

A) 5
B) 10
C) Compilation error
D) 0

Answer: (B)

Explanation: The // starts a comment, so '/ comment */2' is ignored; the expression becomes 'int a = 10;'.*

Q40. Can comments span across preprocessor directives?

A) Yes, always
B) No, preprocessor directives must be on one line
C) Yes, with multi-line comments only
D) Yes, with single-line comments only

Answer: (C)

Explanation: A block comment can span around preprocessor directives in theory, but single-line comments end at the newline and cannot wrap a directive.

Q41. What is the effect of '//' appearing after '/*' but before '*/'?

A) Starts a nested comment
B) Has no special effect; still inside block comment
C) Ends the block comment
D) Causes a compiler warning

Answer: (B)

Explanation: Once inside / */, all content including '//' is treated as part of the block comment.*

Q42. The C standard that formally defines comment handling is:

A) POSIX
B) ISO/IEC 9899
C) IEEE 754
D) RFC 2616

Answer: (B)

Explanation: ISO/IEC 9899 is the official C language standard which defines all aspects of the language including comment handling.

Q43. What does gcc do with the following? /* int main(){ return 0; } */

A) Compiles the main function
B) Produces a 'no main function' error or warning (if required)
C) Ignores it and compiles successfully (no main needed)
D) Produces an executable with empty main

Answer: (B)

Explanation: The code is commented out, so if main is required, the linker/compiler reports it missing.

SYNTAX & TRICKY QUESTIONS

Q44. What is the output of the following program? #include int main(){ int x=10; // x=20; printf("%d",x); }

A) 20
B) 10
C) 0
D) Compilation error

Answer: (B)

Explanation: x=20 is commented out, so x is still 10.

Q45. Identify the error: int a = 5; /* first comment /* nested */ second comment */

A) No error
B) Nested comment causes */ to close prematurely
C) Single-line comment error
D) Variable name error

Answer: (B)

*Explanation: The first */ closes the comment at 'nested', so ' second comment */' is outside the comment and causes a syntax error.*

Q46. What is the output? printf("Rate = %d%%", 50 /* percent */);

A) Rate = 50%
B) Rate = 50%%
C) Compilation error
D) Rate = 50 percent

Answer: (A)

Explanation: / percent */ is a comment and is removed; %% in printf prints a literal %, so output is 'Rate = 50%'.*

Q47. Which of the following is correctly commented? A) int x = 5; // Set x to 5 B) int x = 5; /* Set x /* to */ 5 */ C) int x = 5; /* Set x to 5 D) int /* x */ = 5;

A) Only A
B) A and D
C) A, B, and D
D) All of A, B, C, D

Answer: (B)

Explanation: A is a valid single-line comment. D is valid (comments can appear between tokens). B has a nested comment issue. C has an unclosed comment.

Q48. What will be the value of z? int z = 4 + /* 3 + */ 2;

A) 9
B) 6
C) 7
D) 4

Answer: (B)

Explanation: The comment / 3 + */ is removed, leaving int z = 4 + 2; so z = 6.*

Q49. Which of the following programs will compile without error? A) int main(){/*}*/} B) int main(){/*} */} C) int main(){/* hello */} D) All of the above

A) Only C
B) A and C
C) B and C
D) All of A, B, C

Answer: (C)

Explanation: In A, the / is closed by the */ inside the string-like sequence, and then a stray } causes error. C is clearly correct. B closes the comment before }, so it compiles. D is incorrect.*

Q50. The following code: int y = 1//* divide */0 + 1; evaluates y as:

A) Compilation error (division by zero)
B) 2
C) 1
D) 0

Answer: (C)

Explanation: The '//' starts a single-line comment eating '/ divide */0 + 1', so expression is int y = 1; giving y=1.*

Q51. Comments in C can appear:

A) Only before statements
B) Only after statements
C) Anywhere whitespace is allowed
D) Only at the beginning of lines

Answer: (C)

Explanation: Comments can appear anywhere whitespace is permissible in C source code.

Q52. What is the output? // This is\ a multi-line comment printf("Hi");

A) Hi
B) Nothing
C) Compilation error
D) a multi-line comment

Answer: (B)

Explanation: The backslash at the end of the // comment line continues the comment onto the next line (line splicing), so printf is also commented out.

Q53. Which of the following statements about comments is FALSE?

A) Comments are removed before compilation
B) Comments can include any characters
C) Comments improve runtime performance
D) Comments can span multiple lines using /* */

Answer: (C)

Explanation: Comments have NO effect on runtime performance; they are removed entirely before the binary is generated.

Q54. What is the output? #include int main(){ int /*a*/ b = 5; printf("%d",b); }

A) Compilation error
B) Garbage value
C) 5
D) 0

Answer: (C)

*Explanation: The comment /*a*/ between int and b is valid; b is declared as int and assigned 5.*

Q55. Which is true about '//' in C89?

A) It is a valid single-line comment
B) It is not recognised as a comment
C) It causes undefined behaviour
D) It is a division operator twice

Answer: (B)

Explanation: C89/ANSI C does not support // single-line comments. They were added only in C99.

Q56. How does the compiler treat: int a = 10; /* comment int b = 20; */ printf("%d",a);

A) Prints 10
B) Prints 20
C) Compilation error
D) Prints 10 then 20

Answer: (A)

Explanation: The block comment spans two lines, commenting out 'int b = 20;'. a = 10 is valid and printed.

Q57. In which phase of compilation are comments processed?

A) Phase 6 (Translation)
B) Phase 3 (Tokenisation)
C) Phase 7 (Linking)
D) Phase 1 (Character mapping)

Answer: (B)

Explanation: According to the C standard translation phases, comments are handled in Phase 3 (tokenisation/lexical analysis).

ERROR DETECTION QUESTIONS

Q58. Find the error: /* This is a comment /

A) No error
B) Comment not properly closed
C) Wrong opening symbol
D) Too long

Answer: (B)

Explanation: The comment is opened with / but closed with / instead of */, so it is not properly closed.*

Q59. Find the error: int main(){ printf("Hello"); // return 0 }

A) printf is missing a semicolon
B) return 0 is commented out (may cause warning)
C) main must return int explicitly
D) No error

Answer: (B)

Explanation: While returning from main without return 0 is legal in C99+, commenting it out may cause compiler warnings in strict mode.

Q60. What error does the following contain? /* int x = 5; /* inner */ int y = 10; */

A) No error
B) Nested /* causes premature comment close
C) Variables in comments are illegal
D) Too many lines in comment

Answer: (B)

Explanation: The inner / is fine but the first */ closes the outer comment; 'int y = 10;' is now live code followed by a lone */, causing a syntax error.*

Q61. Identify the bug: #define MAX 100 // max value #define MIN 0 /* min value

A) MAX definition error
B) MIN comment not closed
C) Cannot mix // and /* */
D) No bug

Answer: (B)

Explanation: / min value is opened but never closed, so everything after it to end-of-file is treated as a comment.*

Q62. Is the following valid C code? int main(){int/*comment*/x=5;return 0;}

A) No, comment cannot appear inside a declaration
B) Yes, it is valid
C) Only valid in C11
D) Causes undefined behaviour

Answer: (B)

*Explanation: A comment can appear between any two tokens; int/*comment*/x=5 is equivalent to int x=5.*

Q63. What is wrong with: printf("Value: %d", /* 42 */);

A) Nothing
B) Missing argument for %d after comment removal
C) printf syntax error
D) Comment inside printf is illegal

Answer: (B)

Explanation: After removing / 42 */, printf has no argument for %d, causing undefined behaviour or a warning.*

Q64. Spot the error: for(int i=0; /* i<10 */ i++){...}

A) for loop syntax error
B) The condition is commented out, loop runs forever or not at all
C) i++ is wrong
D) No error

Answer: (B)

Explanation: The loop condition / i<10 */ is a comment, making the condition empty, which is treated as 'true' in C — infinite loop.*

Q65. What will gcc report for: int a = 5 / 0; // legal at runtime?

A) Warning: division by zero
B) Silent; division by zero only occurs at runtime
C) The comment causes the error
D) Compilation succeeds silently

Answer: (A)

Explanation: GCC typically warns about division by zero at compile time (constant folding). The comment has no effect on this.

ADVANCED & MIXED QUESTIONS

Q66. Which of the following is a valid technique to temporarily remove a block of code in C?

A) Delete it
B) Wrap it in /* */
C) Use #if 0 ... #endif
D) Both B and C

Answer: (D)

Explanation: Both wrapping in / */ and using #if 0 ... #endif are common techniques for temporarily disabling code.*

Q67. Why is #if 0 ... #endif preferred over /* */ for commenting out large code blocks?

A) It is faster to type
B) /* */ does not support nesting; #if 0 does
C) It produces smaller executables
D) It is the C89 standard

Answer: (B)

Explanation: #if 0 properly nests even if the code contains / */ comments, whereas using /* */ on code that already has /* */ causes premature termination.*

Q68. What does the following output? #include int main(){ int a=1,b=2; // a=3; /* b=4; */ printf("%d %d",a,b); }

A) 3 4
B) 1 2
C) 3 2
D) 1 4

Answer: (B)

Explanation: Both assignments are inside comments, so a and b retain their values 1 and 2.

Q69. Which of the following is true? I. Comments can be used inside string literals II. Comments can be used between function arguments III. Comments can span across lines using //

A) Only I
B) Only II
C) I and II
D) II only; I is false; III is false

Answer: (D)

Explanation: I is false (// or / inside a string is literal text). II is true (e.g., func(/*arg1*/5)). III is false (// only comments to end of line).*

Q70. The output of the following is: int x = 5; printf("%d", x /*+5*/ +10);

A) 20
B) 15
C) 10
D) Compilation error

Answer: (B)

Explanation: /+5*/ is removed, leaving printf("%d", x +10); → 5+10 = 15.*

Q71. Which C compiler flag shows warnings for C89 code where // comments appear?

A) -Wall
B) -std=c89 -pedantic
C) -O2
D) -g

Answer: (B)

Explanation: Using '-std=c89 -pedantic' tells GCC to enforce strict C89 rules, where // comments are not standard.

Q72. What is the output? char *s = "Hello //World"; printf("%s", s);

A) Hello
B) Hello //World
C) Hello World
D) Compilation error

Answer: (B)

Explanation: The // inside a string literal is not a comment; the full string is printed.

Q73. How many lines will the following block comment span? /* Line1 Line2 Line3 */

A) 1
B) 3
C) 5 (including /* and */)
D) Depends on compiler

Answer: (C)

Explanation: The block comment includes the opening /, three content lines, and the closing */ — 5 lines in total.*

Q74. What technique uses comments to document function interfaces?

A) MISRA
B) Javadoc-style comments (e.g., Doxygen)
C) assert()
D) #pragma

Answer: (B)

*Explanation: Doxygen-style comments (/**...*/) are used to document function parameters, return values, and descriptions.*

Q75. Which is true about comments and macros? A) Comments inside #define body are replaced B) /* */ on the same line as a macro definition may end the macro line C) // comments terminate the macro definition D) All of the above

A) A only
B) B only
C) C only
D) A and C

Answer: (D)

Explanation: A: Comments inside #define are processed. C: A // comment on the same line as #define ends the macro (since macros are single-line). B is generally incorrect.

Q76. What happens here? int x=5; /* int y=10; */ int z=15;

A) z is not declared
B) All three variables declared
C) Only x and z are declared
D) Compilation error

Answer: (C)

Explanation: y=10 is inside the block comment and is not a declaration; only x and z are declared.

Q77. Which output is produced? // printf("A"); printf("B"); // printf("C");

A) A B C
B) B
C) ABC
D) Nothing

Answer: (B)

Explanation: The first and third printf calls are commented out; only printf("B") executes.

Q78. The line '//\' (backslash at end of // comment) in C:

A) Has no special effect
B) Continues the comment on the next line (line splicing)
C) Causes a compilation error
D) Only valid in C11

Answer: (B)

Explanation: A backslash at the end of a line (line continuation) causes the next line to be treated as a continuation of the comment.

Q79. In C, which type of comment is NOT recommended inside macro definitions?

A) //
B) /* */
C) Both
D) Neither; both are fine in macros

Answer: (A)

Explanation: // comments end at the newline; if a macro uses line continuation (\), a // comment may accidentally comment out part of the macro. / */ is safer in macros.*

Q80. Can you use a comment to separate the name of a variable from its type? Example: int/*space*/x=5;

A) No, causes error
B) Yes, it compiles
C) Only in C99
D) Only in C11

Answer: (B)

*Explanation: Comments can replace whitespace between any two tokens in C, so int/*space*/x=5 is valid.*

Q81. What does the following output? int a=2,b=3; int c=a/* +b */; printf("%d",c);

A) 5
B) 2
C) 3
D) Compilation error

Answer: (B)

Explanation: / +b */ is a comment; the expression becomes c = a; so c = 2.*

Q82. Which of the following about comments is FALSE?

A) They improve code maintainability
B) They are part of the object code
C) They can be placed between function arguments
D) They are removed before code generation

Answer: (B)

Explanation: Comments are never part of the object/binary code; they are stripped before code generation.

Q83. In a header file (.h), comments are used mainly for:

A) Storing variable values
B) Documenting function prototypes and macros
C) Controlling compilation
D) Linking purposes

Answer: (B)

Explanation: Header files use comments to document APIs: function signatures, parameter descriptions, and return values.

Q84. What is the maximum length of a single-line comment in C?

A) 80 characters
B) 255 characters
C) 512 characters
D) No standard limit

Answer: (D)

Explanation: The C standard does not impose a limit on comment length; practical limits depend on the compiler's line-length handling.

Q85. Which of the following is TRUE about comments and conditional compilation?

A) Comments override #ifdef conditions
B) Code inside comments is never compiled regardless of #ifdef
C) #ifdef can enable comments
D) Comments are processed after conditional compilation

Answer: (B)

Explanation: Code inside comments is always ignored by the compiler, no matter what preprocessor conditions surround it.

Q86. What is printed? #include int main(){ int i; for(i=0;i<3;i++) //i++; printf("%d",i); return 0; }

A) 0
B) 1
C) 3
D) Infinite loop — nothing printed

Answer: (D)

Explanation: The loop body is the single-line comment '//i++;', not an empty statement. With an empty body for(i=0;i<3;i++) it runs and exits... actually: the for has no body statement here (just the // comment), so it loops 3 times normally with i becoming 3. Wait — the for body is empty (// comment), so i increments to 3 and stops. Output: 3.

Q87. What will the code output? (Corrected Q87) for(i=0;i<3;i++) /* i+=2; */; printf("%d",i);

A) 0
B) 3
C) 6
D) Infinite loop

Answer: (B)

Explanation: The comment removes i+=2; and the ; is an empty statement body. The loop runs i=0,1,2 and then i=3 exits. Output: 3.

Q88. A comment placed just before a function definition is called:

A) Inline comment
B) Function header comment
C) Block comment
D) Trailing comment

Answer: (B)

Explanation: A comment placed immediately before a function definition describing its purpose is traditionally called a function header comment.

Q89. Which of the following characters CANNOT start a comment in C?

A) /
B) *
C) #
D) Both B and C

Answer: (D)

Explanation: In C, only '/' is the leading character for comments (// or /). '#' starts preprocessor directives, and '*' alone does not start a comment.*

Q90. What is the output of the program? int a=10; int b=20; //int c=a+b; printf("%d",a+b);

A) 30
B) 0
C) Compilation error
D) Undefined

Answer: (A)

Explanation: Only 'int c=a+b;' is commented out; a+b in printf is live code, producing 30.

Q91. Which of the following correctly describes 'dead code comments'?

A) Comments about infinite loops
B) Commenting out code that will never execute
C) Comments after return statements
D) Both B and C

Answer: (D)

Explanation: Dead code refers to code that never executes; commenting it out or leaving comments after unreachable return statements both qualify.

Q92. In a multi-file C project, comments are most important in:

A) .c implementation files only
B) .h header files only
C) Both .c and .h files
D) Only the main.c file

Answer: (C)

Explanation: Good commenting practice requires documentation in both header files (API contracts) and source files (implementation details).

Q93. The following code has how many comments? int x=1; /*c1*/ int y=2; //c2 int z=3; /*c3 c4*/

A) 2
B) 3
C) 4
D) 1

Answer: (B)

*Explanation: There are 3 comment tokens: /*c1*/, //c2, and /*c3\nc4*/ (one block comment spanning two lines).*

Q94. What best practice says about comments and code?

A) Comments should outnumber lines of code
B) Comments should explain WHY, not WHAT
C) Every line needs a comment
D) Comments should be in uppercase

Answer: (B)

Explanation: Best practice: the code itself shows WHAT; comments should explain WHY a decision was made, especially for non-obvious logic.

Q95. Which of the following is a documentation comment convention for C?

A) ///
B) /**
C) ##
D) //!

Answer: (B)

*Explanation: Doxygen uses /** ... */ (starting with double asterisk) as its documentation comment format in C.*

Q96. What is printed? printf("/* not a comment */");

A) (nothing)
B) /* not a comment */
C) Compilation error
D) not a comment

Answer: (B)

Explanation: Inside a string literal, / ... */ is not treated as a comment; it is printed literally.*

Q97. A comment is considered a:

A) Token in C
B) White-space equivalent after preprocessing
C) Keyword
D) Operator

Answer: (B)

Explanation: After preprocessing, a comment is replaced by a single space and is thus treated as whitespace by the compiler.

Q98. Which of the following pairs are BOTH valid C comment styles?

A) // and ##
B) // and /* */
C) # and /* */
D) // and

Answer: (B)

Explanation: // (C99+) and / */ (C89+) are the only two valid comment styles in C.*

Q99. What happens when /* appears inside a // comment?

A) Starts a block comment
B) Has no effect; remains part of the single-line comment
C) Causes a compilation error
D) Ends the line comment early

Answer: (B)

Explanation: Once // starts a single-line comment, all characters including / are part of that comment until end of line.*

Q100. Which of the following is the best use of a single-line comment?

A) Commenting out large blocks of code
B) Short explanatory notes at end of a statement
C) Function documentation headers
D) Multi-paragraph algorithm explanations

Answer: (B)

Explanation: Single-line // comments are best suited for brief annotations at the end of or above a line of code.

Q101. The output of the following is: int x = 5 + /* 3 */ - 2; printf("%d", x);

A) 6
B) 3
C) 10
D) Compilation error

Answer: (B)

Explanation: / 3 */ is removed, leaving x = 5 + - 2 = 5 - 2 = 3.*

Q102. How does a block comment affect token parsing? int a/*comment*/=5;

A) Error: token split
B) Valid: comment acts as whitespace between tokens
C) Only valid in C11
D) Produces undefined behaviour

Answer: (B)

*Explanation: Block comments act as whitespace; 'a/*comment*/=' is equivalent to 'a ='.*

Q103. Which of the following about comments and the linker is TRUE?

A) Linker reads comments to resolve symbols
B) Comments are completely absent from object files
C) Linker uses comments for optimisation
D) Debug comments are preserved by the linker

Answer: (B)

Explanation: Comments are stripped during preprocessing, long before linking. Object files contain no comments.

Q104. What is the best reason to avoid over-commenting?

A) Comments slow down compilation
B) Excessive comments can obscure code structure and become outdated
C) Comments use extra memory at runtime
D) Compilers limit comments to 1000

Answer: (B)

Explanation: Too many comments — especially redundant ones — can clutter the code and become misleading when the code changes but comments are not updated.

Q105. A comment that explains WHY a workaround is used is called:

A) Redundant comment
B) Explanatory comment
C) TODO comment
D) Inline comment

Answer: (B)

Explanation: An explanatory comment documents the reasoning (WHY) behind non-obvious code decisions.

Q106. In MISRA-C guidelines, which comment style is recommended?

A) // only
B) /* */ only
C) Both equally
D) No comments allowed

Answer: (B)

Explanation: MISRA-C (for safety-critical systems) recommends only / */ style comments, not //, for portability with C89 compilers.*

Q107. What does the following output? int a = 1; if (a == 1) // comment printf("Yes"); else printf("No");

A) Yes
B) No
C) Compilation error
D) Nothing

Answer: (A)

Explanation: The comment after 'if (a==1)' has no effect; printf("Yes") is the if-body (no braces needed for single statement).

Q108. Which of the following C programs has NO comments?

A) int main(){return 0;/* end */}
B) int main(){return 0;}
C) int main(){//done return 0;}
D) int/*main*/main(){return 0;}

Answer: (B)

Explanation: Only option B has no comment syntax at all.

Q109. The 'commenting style guide' for a project primarily ensures:

A) Faster compilation
B) Consistency and readability across the codebase
C) Smaller binary size
D) Fewer runtime errors

Answer: (B)

Explanation: A style guide for comments ensures all developers use a uniform format, making the codebase easier to read and maintain.

PRACTICE TEST 1

Questions: 1–25 (25 questions) **Time Allowed:** 30 minutes

Coverage: Covers: Single-line & Multi-line Comments, Uses of Comments

Instructions: Each question carries 1 mark. There is NO negative marking. Choose the BEST answer from the four options. Write only the letter (A/B/C/D) in the blank provided.

SINGLE-LINE COMMENTS

Q1. Which symbol is used to begin a single-line comment in C?

A) #
B) //
C) /*
D) --

Q2. What is the scope of a single-line comment in C?

A) Until the next semicolon
B) Until the closing */,
C) Until the end of the current line
D) Until the end of the function

Q3. Which of the following is a valid single-line comment in C?

A) /* This is a comment */
B) // This is a comment
C) # This is a comment
D) -- This is a comment

Q4. What will be the output of the following code? #include int main(){ int x=5; //x=10; printf("%d",x); return 0; }

A) 10
B) 5
C) Compilation error
D) Undefined behaviour

Q5. Single-line comments in C were introduced in which standard?

A) C89/ANSI C
B) C99
C) C11
D) C17

Q6. Which of the following is NOT a valid way to comment out a single line in C?

A) // int x = 5;
B) /* int x = 5; */
C) # int x = 5;
D) Both B and C are valid

Q7. In C, can a single-line comment appear after a statement on the same line?

A) No
B) Yes
C) Only in C++
D) Only in C11

Q8. What happens if '//' appears inside a string literal?

A) It starts a comment
B) It is treated as part of the string
C) Compilation error
D) Undefined behaviour

Q9. How many single-line comments can appear in one C program?

A) Maximum 100
B) Maximum 255
C) Unlimited
D) Only 1 per function

Q10. What will the following print? printf("Hello // World");

A) Hello
B) Hello // World
C) Nothing after //
D) Compilation error

Q11. Which of the following correctly places a single-line comment?

A) int a = 5 // value;
B) int a = //5; 5;
C) //int a = 5;
D) Both A and C

MULTI-LINE COMMENTS

Q12. Which symbols are used to start and end a multi-line comment in C?

A) // and //
B) /* and */
C) { and }
D)

Q13. A multi-line comment in C is also called a:

A) Shell comment
B) Block comment
C) Inline comment
D) Preprocessor comment

Q14. Can multi-line comments be nested in C?

A) Yes, unlimited nesting
B) Yes, up to 10 levels
C) No, they cannot be nested
D) Yes, but only in C11

Q15. What is the output of the following code? int x = /* 5 */ 10; printf("%d", x);

A) 5
B) 10
C) 55
D) Compilation error

Q16. Which of the following multi-line comment structures will cause an error in C?

A) /* Hello World */
B) /* Hello World */
C) /* Hello /* inner */ World */
D) /* */

Q17. What happens if a multi-line comment is not closed?

A) The rest of the file is treated as comment
B) Only the current function is affected
C) The compiler ignores the error
D) A warning is issued but code compiles

Q18. Which standard first included multi-line comments (/* */) in C?

A) C99
B) C11
C) C89 (ANSI C)
D) C17

Q19. A multi-line comment can span how many lines?

A) Up to 5 lines
B) Up to 100 lines
C) Any number of lines
D) Up to the file size limit

Q20. What will be the output? /* int x=5; int y=10; */ printf("Hello");

A) 5 10 Hello
B) Hello
C) Compilation error
D) Undefined

Q21. Which of the following is a valid multi-line comment?

A) /* This is line1 This is line2 */
B) // This is line1 // This is line2
C) Both A and B
D) Neither A nor B

Q22. What is the minimum valid multi-line comment in C?

A) /
B) /**/
C) /* */
D) Both B and C

Q23. Inside a multi-line comment, what happens to the code written in it?

A) It is executed
B) It is ignored by the compiler
C) It causes warnings
D) It is stored in memory

USES OF COMMENTS

Q24. Which of the following is the primary purpose of comments in C?

A) To increase execution speed
B) To document and explain code
C) To allocate memory
D) To define macros

Q25. Commenting out code during debugging is used to:

A) Permanently delete code
B) Temporarily disable code without deleting it
C) Speed up compilation
D) Allocate extra memory

PRACTICE TEST 2

Questions: 26–50 (25 questions) **Time Allowed:** 30 minutes

Coverage: Covers: Compiler Behaviour, Syntax & Tricky Questions

Instructions: Each question carries 1 mark. There is NO negative marking. Choose the BEST answer from the four options. Write only the letter (A/B/C/D) in the blank provided.

USES OF COMMENTS

Q26. Which comment style is preferred for function documentation in C?

A) Single-line //
B) Block comment /* ... */
C) Both are equally preferred
D) Neither; documentation is separate

Q27. Comments in C are processed by:

A) The linker
B) The loader
C) The preprocessor/compiler (and stripped out)
D) The CPU

Q28. Which of the following is NOT a valid use of comments in C?

A) Describing the algorithm
B) Hiding sensitive data at runtime
C) Noting author and date
D) Explaining complex logic

Q29. Using comments to explain the purpose of a variable is called:

A) Debugging
B) Inline documentation
C) Preprocessing
D) Profiling

Q30. Which tool commonly uses specially formatted comments to auto-generate documentation?

A) make
B) Doxygen
C) gdb
D) valgrind

Q31. What is a 'TODO comment'?

A) A comment marking code to delete
B) A comment noting a task still to be done
C) A compiler directive
D) A preprocessor command

Q32. Comments are often used to preserve:

A) Old or alternate versions of code
B) Variable values at runtime
C) Stack frames
D) File headers only

Q33. Which of the following best describes self-documenting code?

A) Code with maximum comments
B) Code with clear variable/function names reducing the need for comments
C) Code without any comments
D) Code generated automatically

COMPILER BEHAVIOUR WITH COMMENTS

Q34. At which stage does the C compiler remove comments?

A) Linking
B) Assembly
C) Preprocessing/Lexical analysis
D) Code optimisation

Q35. After removing a comment, the compiler replaces it with:

A) Nothing (empty string)
B) A single space character
C) A newline
D) A null byte

Q36. What does the compiler do with the text inside a comment?

A) Compiles it separately
B) Ignores it completely
C) Generates warnings about it
D) Stores it in the symbol table

Q37. Do comments affect the size of the compiled (binary) executable?

A) Yes, they increase it
B) Yes, they decrease it
C) No, they are stripped before compilation
D) Depends on compiler flags

Q38. Which of the following will cause a compilation error related to comments?

A) // unclosed comment
B) /* unclosed comment
C) // nested // comment
D) /* empty comment */

Q39. What is the output? int a = 10//* comment */2; printf("%d",a);

A) 5
B) 10
C) Compilation error
D) 0

Q40. Can comments span across preprocessor directives?

A) Yes, always
B) No, preprocessor directives must be on one line
C) Yes, with multi-line comments only
D) Yes, with single-line comments only

Q41. What is the effect of '//' appearing after '/*' but before '*/'?

A) Starts a nested comment
B) Has no special effect; still inside block comment
C) Ends the block comment
D) Causes a compiler warning

Q42. The C standard that formally defines comment handling is:

A) POSIX
B) ISO/IEC 9899
C) IEEE 754
D) RFC 2616

Q43. What does gcc do with the following? /* int main(){ return 0; } */

A) Compiles the main function
B) Produces a 'no main function' error or warning (if required)
C) Ignores it and compiles successfully (no main needed)
D) Produces an executable with empty main

SYNTAX & TRICKY QUESTIONS

Q44. What is the output of the following program? #include int main(){ int x=10; // x=20; printf("%d",x); }

A) 20
B) 10
C) 0
D) Compilation error

Q45. Identify the error: int a = 5; /* first comment /* nested */ second comment */

A) No error
B) Nested comment causes */ to close prematurely
C) Single-line comment error
D) Variable name error

Q46. What is the output? printf("Rate = %d%%", 50 /* percent */);

A) Rate = 50%
B) Rate = 50%%
C) Compilation error
D) Rate = 50 percent

Q47. Which of the following is correctly commented? A) int x = 5; // Set x to 5 B) int x = 5; /* Set x /* to */ 5 */ C) int x = 5; /* Set x to 5 D) int /* x */ = 5;

A) Only A
B) A and D
C) A, B, and D
D) All of A, B, C, D

Q48. What will be the value of z? int z = 4 + /* 3 + */ 2;

A) 9
B) 6
C) 7
D) 4

Q49. Which of the following programs will compile without error? A) int main(){/*}*/} B) int main(){/*} */} C) int main(){/* hello */} D) All of the above

A) Only C
B) A and C
C) B and C
D) All of A, B, C

Q50. The following code: int y = 1//* divide */0 + 1; evaluates y as:

A) Compilation error (division by zero)
B) 2
C) 1
D) 0

PRACTICE TEST 3

Questions: 51–75 (25 questions) **Time Allowed:** 30 minutes

Coverage: Covers: Error Detection & Advanced Mixed Questions

Instructions: Each question carries 1 mark. There is NO negative marking. Choose the BEST answer from the four options. Write only the letter (A/B/C/D) in the blank provided.

SYNTAX & TRICKY QUESTIONS

Q51. Comments in C can appear:

A) Only before statements
B) Only after statements
C) Anywhere whitespace is allowed
D) Only at the beginning of lines

Q52. What is the output? // This is\ a multi-line comment printf("Hi");

A) Hi
B) Nothing
C) Compilation error
D) a multi-line comment

Q53. Which of the following statements about comments is FALSE?

A) Comments are removed before compilation
B) Comments can include any characters
C) Comments improve runtime performance
D) Comments can span multiple lines using /* */

Q54. What is the output? #include int main(){ int /*a*/ b = 5; printf("%d",b); }

A) Compilation error
B) Garbage value
C) 5
D) 0

Q55. Which is true about '//' in C89?

A) It is a valid single-line comment
B) It is not recognised as a comment
C) It causes undefined behaviour
D) It is a division operator twice

Q56. How does the compiler treat: int a = 10; /* comment int b = 20; */ printf("%d",a);

A) Prints 10
B) Prints 20
C) Compilation error
D) Prints 10 then 20

Q57. In which phase of compilation are comments processed?

A) Phase 6 (Translation)
B) Phase 3 (Tokenisation)
C) Phase 7 (Linking)
D) Phase 1 (Character mapping)

ERROR DETECTION QUESTIONS

Q58. Find the error: /* This is a comment /

A) No error
B) Comment not properly closed
C) Wrong opening symbol
D) Too long

Q59. Find the error: int main(){ printf("Hello"); // return 0 }

A) printf is missing a semicolon
B) return 0 is commented out (may cause warning)
C) main must return int explicitly
D) No error

Q60. What error does the following contain? /* int x = 5; /* inner */ int y = 10; */

A) No error
B) Nested /* causes premature comment close
C) Variables in comments are illegal
D) Too many lines in comment

Q61. Identify the bug: #define MAX 100 // max value #define MIN 0 /* min value

A) MAX definition error
B) MIN comment not closed
C) Cannot mix // and /* */
D) No bug

Q62. Is the following valid C code? int main(){int/*comment*/x=5;return 0;}

A) No, comment cannot appear inside a declaration
B) Yes, it is valid
C) Only valid in C11
D) Causes undefined behaviour

Q63. What is wrong with: printf("Value: %d", /* 42 */);

A) Nothing
B) Missing argument for %d after comment removal
C) printf syntax error
D) Comment inside printf is illegal

Q64. Spot the error: for(int i=0; /* i<10 */ i++){...}

A) for loop syntax error
B) The condition is commented out, loop runs forever or not at all
C) i++ is wrong
D) No error

Q65. What will gcc report for: int a = 5 / 0; // legal at runtime?

A) Warning: division by zero
B) Silent; division by zero only occurs at runtime
C) The comment causes the error
D) Compilation succeeds silently

ADVANCED & MIXED QUESTIONS

Q66. Which of the following is a valid technique to temporarily remove a block of code in C?

A) Delete it
B) Wrap it in /* */
C) Use #if 0 ... #endif
D) Both B and C

Q67. Why is #if 0 ... #endif preferred over /* */ for commenting out large code blocks?

A) It is faster to type
B) /* */ does not support nesting; #if 0 does
C) It produces smaller executables
D) It is the C89 standard

Q68. What does the following output? #include int main(){ int a=1,b=2; // a=3; /* b=4; */ printf("%d %d",a,b); }

A) 3 4
B) 1 2
C) 3 2
D) 1 4

Q69. Which of the following is true? I. Comments can be used inside string literals II. Comments can be used between function arguments III. Comments can span across lines using //

A) Only I
B) Only II
C) I and II
D) II only; I is false; III is false

Q70. The output of the following is: int x = 5; printf("%d", x /*+5*/ +10);

A) 20
B) 15
C) 10
D) Compilation error

Q71. Which C compiler flag shows warnings for C89 code where // comments appear?

A) -Wall
B) -std=c89 -pedantic
C) -O2
D) -g

Q72. What is the output? char *s = "Hello //World"; printf("%s", s);

A) Hello
B) Hello //World
C) Hello World
D) Compilation error

Q73. How many lines will the following block comment span? /* Line1 Line2 Line3 */

A) 1
B) 3
C) 5 (including /* and */)
D) Depends on compiler

Q74. What technique uses comments to document function interfaces?

A) MISRA
B) Javadoc-style comments (e.g., Doxygen)
C) assert()
D) #pragma

Q75. Which is true about comments and macros? A) Comments inside #define body are replaced B) /* */ on the same line as a macro definition may end the macro line C) // comments terminate the macro definition D) All of the above

A) A only
B) B only
C) C only
D) A and C

PRACTICE TEST 4

Questions: 76–110 (35 questions) **Time Allowed:** 35 minutes

Coverage: Covers: Full Mixed Review — All Topics

Instructions: Each question carries 1 mark. There is NO negative marking. Choose the BEST answer from the four options. Write only the letter (A/B/C/D) in the blank provided.

ADVANCED & MIXED QUESTIONS

Q76. What happens here? int x=5; /* int y=10; */ int z=15;

A) z is not declared
B) All three variables declared
C) Only x and z are declared
D) Compilation error

Q77. Which output is produced? // printf("A"); printf("B"); // printf("C");

A) A B C
B) B
C) ABC
D) Nothing

Q78. The line '//\' (backslash at end of // comment) in C:

A) Has no special effect
B) Continues the comment on the next line (line splicing)
C) Causes a compilation error
D) Only valid in C11

Q79. In C, which type of comment is NOT recommended inside macro definitions?

A) //
B) /* */
C) Both
D) Neither; both are fine in macros

Q80. Can you use a comment to separate the name of a variable from its type? Example: int/*space*/x=5;

A) No, causes error
B) Yes, it compiles
C) Only in C99
D) Only in C11

Q81. What does the following output? int a=2,b=3; int c=a/* +b */; printf("%d",c);

A) 5
B) 2
C) 3
D) Compilation error

Q82. Which of the following about comments is FALSE?

A) They improve code maintainability
B) They are part of the object code
C) They can be placed between function arguments
D) They are removed before code generation

Q83. In a header file (.h), comments are used mainly for:

A) Storing variable values
B) Documenting function prototypes and macros
C) Controlling compilation
D) Linking purposes

Q84. What is the maximum length of a single-line comment in C?

A) 80 characters
B) 255 characters
C) 512 characters
D) No standard limit

Q85. Which of the following is TRUE about comments and conditional compilation?

A) Comments override #ifdef conditions
B) Code inside comments is never compiled regardless of #ifdef
C) #ifdef can enable comments
D) Comments are processed after conditional compilation

Q86. What is printed? #include int main(){ int i; for(i=0;i<3;i++) //i++; printf("%d",i); return 0; }

A) 0
B) 1
C) 3
D) Infinite loop — nothing printed

Q87. What will the code output? (Corrected Q87) for(i=0;i<3;i++) /* i+=2; */; printf("%d",i);

A) 0
B) 3
C) 6
D) Infinite loop

Q88. A comment placed just before a function definition is called:

A) Inline comment
B) Function header comment
C) Block comment
D) Trailing comment

Q89. Which of the following characters CANNOT start a comment in C?

A) /
B) *
C) #
D) Both B and C

Q90. What is the output of the program? int a=10; int b=20; //int c=a+b; printf("%d",a+b);

A) 30
B) 0
C) Compilation error
D) Undefined

Q91. Which of the following correctly describes 'dead code comments'?

A) Comments about infinite loops
B) Commenting out code that will never execute
C) Comments after return statements
D) Both B and C

Q92. In a multi-file C project, comments are most important in:

A) .c implementation files only
B) .h header files only
C) Both .c and .h files
D) Only the main.c file

Q93. The following code has how many comments? int x=1; /*c1*/ int y=2; //c2 int z=3; /*c3 c4*/

A) 2
B) 3
C) 4
D) 1

Q94. What best practice says about comments and code?

A) Comments should outnumber lines of code
B) Comments should explain WHY, not WHAT
C) Every line needs a comment
D) Comments should be in uppercase

Q95. Which of the following is a documentation comment convention for C?

A) ///
B) /**
C) ##
D) //!

Q96. What is printed? printf("/* not a comment */");

A) (nothing)
B) /* not a comment */
C) Compilation error
D) not a comment

Q97. A comment is considered a:

A) Token in C
B) White-space equivalent after preprocessing
C) Keyword
D) Operator

Q98. Which of the following pairs are BOTH valid C comment styles?

A) // and ##
B) // and /* */
C) # and /* */
D) // and

Q99. What happens when /* appears inside a // comment?

A) Starts a block comment
B) Has no effect; remains part of the single-line comment
C) Causes a compilation error
D) Ends the line comment early

Q100. Which of the following is the best use of a single-line comment?

A) Commenting out large blocks of code
B) Short explanatory notes at end of a statement
C) Function documentation headers
D) Multi-paragraph algorithm explanations

Q101. The output of the following is: int x = 5 + /* 3 */ - 2; printf("%d", x);

A) 6
B) 3
C) 10
D) Compilation error

Q102. How does a block comment affect token parsing? int a/*comment*/=5;

A) Error: token split
B) Valid: comment acts as whitespace between tokens
C) Only valid in C11
D) Produces undefined behaviour

Q103. Which of the following about comments and the linker is TRUE?

A) Linker reads comments to resolve symbols
B) Comments are completely absent from object files
C) Linker uses comments for optimisation
D) Debug comments are preserved by the linker

Q104. What is the best reason to avoid over-commenting?

A) Comments slow down compilation
B) Excessive comments can obscure code structure and become outdated
C) Comments use extra memory at runtime
D) Compilers limit comments to 1000

Q105. A comment that explains WHY a workaround is used is called:

A) Redundant comment
B) Explanatory comment
C) TODO comment
D) Inline comment

Q106. In MISRA-C guidelines, which comment style is recommended?

A) // only
B) /* */ only
C) Both equally
D) No comments allowed

Q107. What does the following output? int a = 1; if (a == 1) // comment printf("Yes"); else printf("No");

A) Yes
B) No
C) Compilation error
D) Nothing

Q108. Which of the following C programs has NO comments?

A) int main(){return 0;/* end */}
B) int main(){return 0;}
C) int main(){//done return 0;}
D) int/*main*/main(){return 0;}

Q109. The 'commenting style guide' for a project primarily ensures:

A) Faster compilation
B) Consistency and readability across the codebase
C) Smaller binary size
D) Fewer runtime errors

MASTER ANSWER KEY — ALL SECTIONS (Q1–Q110)

Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans
1	B	23	B	45	B	67	B	89	D
2	C	24	B	46	A	68	B	90	A
3	B	25	B	47	B	69	D	91	D
4	B	26	B	48	B	70	B	92	C
5	B	27	C	49	C	71	B	93	B
6	C	28	B	50	C	72	B	94	B
7	B	29	B	51	C	73	C	95	B
8	B	30	B	52	B	74	B	96	B
9	C	31	B	53	C	75	D	97	B
10	B	32	A	54	C	76	C	98	B
11	D	33	B	55	B	77	B	99	B
12	B	34	C	56	A	78	B	100	B
13	B	35	B	57	B	79	A	101	B
14	C	36	B	58	B	80	B	102	B
15	B	37	C	59	B	81	B	103	B
16	C	38	B	60	B	82	B	104	B
17	A	39	B	61	B	83	B	105	B
18	C	40	C	62	B	84	D	106	B
19	C	41	B	63	B	85	B	107	A
20	B	42	B	64	B	86	D	108	B
21	C	43	B	65	A	87	B	109	B
22	D	44	B	66	D	88	B		

PRACTICE TEST ANSWER KEYS

PRACTICE TEST 1 (Q1–Q25)

Q1: **B** Q2: **C** Q3: **B** Q4: **B** Q5: **B** Q6: **C** Q7: **B** Q8: **B** Q9: **C** Q10: **B**
Q11: **D** Q12: **B** Q13: **B** Q14: **C** Q15: **B** Q16: **C** Q17: **A** Q18: **C** Q19: **C** Q20: **B**
Q21: **C** Q22: **D** Q23: **B** Q24: **B** Q25: **B**

PRACTICE TEST 2 (Q26–Q50)

Q26: **B** Q27: **C** Q28: **B** Q29: **B** Q30: **B** Q31: **B** Q32: **A** Q33: **B** Q34: **C** Q35: **B**
Q36: **B** Q37: **C** Q38: **B** Q39: **B** Q40: **C** Q41: **B** Q42: **B** Q43: **B** Q44: **B** Q45: **B**
Q46: **A** Q47: **B** Q48: **B** Q49: **C** Q50: **C**

PRACTICE TEST 3 (Q51–Q75)

Q51: **C** Q52: **B** Q53: **C** Q54: **C** Q55: **B** Q56: **A** Q57: **B** Q58: **B** Q59: **B** Q60: **B**
Q61: **B** Q62: **B** Q63: **B** Q64: **B** Q65: **A** Q66: **D** Q67: **B** Q68: **B** Q69: **D** Q70: **B**

Q71: **B** Q72: **B** Q73: **C** Q74: **B** Q75: **D**

PRACTICE TEST 4 (Q76–Q110)

Q76: **C** Q77: **B** Q78: **B** Q79: **A** Q80: **B** Q81: **B** Q82: **B** Q83: **B** Q84: **D** Q85: **B**
Q86: **D** Q87: **B** Q88: **B** Q89: **D** Q90: **A** Q91: **D** Q92: **C** Q93: **B** Q94: **B** Q95: **B**
Q96: **B** Q97: **B** Q98: **B** Q99: **B** Q100: **B** Q101: **B** Q102: **B** Q103: **B** Q104: **B** Q105: **B**
Q106: **B** Q107: **A** Q108: **B** Q109: **B**

EXAM TIPS: COMMENTS IN C

1	**Remember C99 Rule** // single-line comments were added in C99. In C89/ANSI C, only /* */ is valid. OAVS exam often tests this distinction.
2	**No Nested Block Comments** /* comments cannot be nested. A */ always closes the nearest preceding /*. Use #if 0...#endif to disable code with existing comments.
3	**Line Splicing with //** A backslash (\) at the end of a // comment line continues the comment to the next line. This is a common tricky question.
4	**Comment in String ≠ Comment** '// text' and '/* text */' inside double quotes are NOT comments — they are part of the string literal.
5	**Comment Replacement** The C standard says a comment is replaced by exactly ONE space, not by nothing. This matters when comments appear between tokens.
6	**Compiler Behaviour** Comments are removed in Translation Phase 3 and do NOT appear in object files, executables, or affect runtime performance.
7	**MISRA-C** Safety-critical C coding standards (MISRA-C) recommend only /* */ and discourage // because of potential line-continuation issues.
8	**Doxygen** Doxygen-style comments start with /** and are used to auto-generate API documentation. Common in professional projects.
9	**Dead Code vs Comments** Best practice: do not leave large blocks of commented-out code in production. Use version control instead.
10	**Exam Strategy** For syntax questions, trace through the comment removal mentally first, then evaluate the remaining expression.

Best of luck with your OAVS Computer Teacher (TGT) examination! Practice consistently and review explanations for questions you get wrong.